In [1]:
```
import numpy as np
import pandas as pd
```

In [2]:
```
np.random.seed(42)
n_records = 1500
```

In [3]:
```
equipment_list = [
    {"id": "EQ-797-01", "model": "Cat 797F Haul Truck", "site": "Kearl Oi
    {"id": "EQ-797-02", "model": "Cat 797F Haul Truck", "site": "Kearl Oi
    {"id": "EQ-390-01", "model": "Cat 390F Excavator", "site": "Highvale
    {"id": "EQ-D11-01", "model": "Cat D11 Dozer", "site": "Fort Hills", "
    {"id": "EQ-994-01", "model": "Cat 994K Wheel Loader", "site": "Red Ch
]
```

In [4]:
```
# 1. Generate Raw IoT Telematics Stream (Complications: Jitter, Nulls, -9
telemetry_data = []
start_date = pd.Timestamp("2026-08-01")

for eq in equipment_list:
    curr_time = start_date
    for _ in range(300):
        # Time jitter (irregular intervals between 35 and 85 mins)
        curr_time += pd.Timedelta(minutes=int(np.random.uniform(35, 85)))

        # Base values
        if eq["unit_sys"] == "Imperial":
            temp = np.random.normal(195, 15)  # Fahrenheit
            press = np.random.normal(2800, 200) # PSI
        else:
            temp = np.random.normal(90, 8)     # Celsius
            press = np.random.normal(193, 14)  # Bar

        # Introduce Complication 2: Sensor Glitches (-9999 or NaN)
        if np.random.rand() < 0.04:
            temp = -9999.0
        elif np.random.rand() < 0.03:
            temp = np.nan

        telemetry_data.append({
            "event_timestamp": curr_time,
            "equipment_id": eq["id"],
            "model": eq["model"],
            "site_location": eq["site"],
            "unit_system": eq["unit_sys"],
            "engine_temp_raw": temp,
            "hydraulic_pressure_raw": press,
            "fuel_rate_lph": np.random.uniform(60, 380) if np.random.rand
            "telematics_status": np.random.choice(["Active", "Buffer_Uplo
        })

df_telemetry = pd.DataFrame(telemetry_data)
```

In [5]:
```
# Introduce Complication: Duplicate Packet Transmissions (Network Burstin
duplicates = df_telemetry.sample(frac=0.03, random_state=42)
df_telemetry = pd.concat([df_telemetry, duplicates]).sort_values("event_t
```

In [6]:
```
# 2. Generate ERP / SAP Maintenance Work Orders (Complication: Unstructur
work_order_notes = [
    "Operator reported high engine temp warning on steep haul road climb.
    "Scheduled 250-hour PM service. Changed engine oil, fuel filters, and
    "Hydraulic pressure drop detected. Replaced worn seal on main boom cy
    "Transmission slippage noted during gear shift 2 to 3. Calibrated clu
    "Routine inspection - no major issues found. Cleared minor sensor fau
]

work_orders = []
for i in range(45):
    eq = np.random.choice(equipment_list)
    wo_start = start_date + pd.Timedelta(days=int(np.random.uniform(1, 50
    downtime_hrs = np.random.uniform(2.5, 18.0)

    work_orders.append({
        "work_order_id": f"WO-2026-{8000 + i}",
        "equipment_id": eq["id"],
        "wo_start_time": wo_start,
        "wo_end_time": wo_start + pd.Timedelta(hours=downtime_hrs),
        "downtime_hours_logged": round(downtime_hrs, 1),
        "maintenance_type": np.random.choice(["Unplanned Breakdown", "Pre
        "technician_notes": np.random.choice(work_order_notes)
    })

df_work_orders = pd.DataFrame(work_orders)
```

In [7]:
```
# Save datasets
df_telemetry.to_csv("raw_telematics_iot.csv", index=False)
df_work_orders.to_csv("sap_work_orders.csv", index=False)

print("Enterprise Complication Datasets Generated:")
print(f"1. raw_telematics_iot.csv ({len(df_telemetry)} records with glitc
print(f"2. sap_work_orders.csv ({len(df_work_orders)} SAP work orders wit
```

```
Enterprise Complication Datasets Generated:
1. raw_telematics_iot.csv (1545 records with glitches, unit mismatches, a
nd duplicates)
2. sap_work_orders.csv (45 SAP work orders with unstructured text)
```

In [8]:
```
#Data Engineering & Governance Pipeline script
```

In [9]:
```
def clean_and_transform_fleet_data():
    print("=== Starting Enterprise Fleet Data Cleansing Pipeline ===")

    # 1. Load Raw Enterprise Datasets
    df_telemetry = pd.read_csv("raw_telematics_iot.csv")
    df_wo = pd.read_csv("sap_work_orders.csv")
```

```
    df_telemetry["event_timestamp"] = pd.to_datetime(
        df_telemetry["event_timestamp"]
    )
    df_wo["wo_start_time"] = pd.to_datetime(df_wo["wo_start_time"])
    df_wo["wo_end_time"] = pd.to_datetime(df_wo["wo_end_time"])

    # ---------------------------------------------------------------------
    # RESOLVING COMPLICATION 1: Remove Duplicate Transmissions (Network B
    # ---------------------------------------------------------------------
    initial_count = len(df_telemetry)
    df_telemetry = df_telemetry.drop_duplicates(
        subset=["equipment_id", "event_timestamp"]
    )
    print(
        f"[✓] Deduplication: Removed {initial_count - len(df_telemetry)}
    )

    # ---------------------------------------------------------------------
    # RESOLVING COMPLICATION 2: Standardize Unit Systems (Imperial -> Met
    # ---------------------------------------------------------------------
    # Fahrenheit to Celsius: (F - 32) * 5/9
    df_telemetry["engine_temp_c"] = np.where(
        df_telemetry["unit_system"] == "Imperial",
        (df_telemetry["engine_temp_raw"] - 32) * 5 / 9,
        df_telemetry["engine_temp_raw"],
    )

    # PSI to Bar: PSI / 14.5038
    df_telemetry["hydraulic_pressure_bar"] = np.where(
        df_telemetry["unit_system"] == "Imperial",
        df_telemetry["hydraulic_pressure_raw"] / 14.5038,
        df_telemetry["hydraulic_pressure_raw"],
    )

    print(
        "[✓] Unit Standardization: Converted all Imperial (Fahrenheit/PSI
    )

    # ---------------------------------------------------------------------
    # RESOLVING COMPLICATION 3: Handle Sensor Anomalies (-9999 & NaNs)
    # ---------------------------------------------------------------------
    # Replace -9999 sentinel glitch values with NaN
    df_telemetry["engine_temp_c"] = df_telemetry["engine_temp_c"].replace
        -9999.0, np.nan
    )

    # Forward-fill / linear interpolate missing values by equipment serie
    df_telemetry["engine_temp_c"] = (
        df_telemetry.groupby("equipment_id")["engine_temp_c"]
        .transform(lambda group: group.interpolate(method="linear"))
        .round(1)
    )

    print(
```

```
        "[✓] Anomaly Handling: Filtered out -9999 sensor glitches and int
    )

    # ---------------------------------------------------------------------
    # RESOLVING COMPLICATION 4: Unstructured Text Parsing (NLP / Regex)
    # ---------------------------------------------------------------------
    def categorize_notes(text):
        text_lower = str(text).lower()
        if "engine" in text_lower or "thermostat" in text_lower:
            return "Engine System"
        elif "hydraulic" in text_lower or "leak" in text_lower:
            return "Hydraulic System"
        elif "transmission" in text_lower or "clutch" in text_lower:
            return "Drivetrain / Transmission"
        elif "service" in text_lower or "pm" in text_lower:
            return "Routine Preventive Maintenance"
        else:
            return "General Inspection"

    df_wo["parsed_component_category"] = df_wo["technician_notes"].apply(
        categorize_notes
    )
    print(
        "[✓] NLP Text Parsing: Extracted structured component categories
    )

    # ---------------------------------------------------------------------
    # RESOLVING COMPLICATION 5: Source Reconciliation (Telematics vs SAP
    # ---------------------------------------------------------------------
    # Check if a telematics record falls within an active SAP Work Order
    df_telemetry["is_in_active_wo_window"] = False

    for _, wo in df_wo.iterrows():
        mask = (
            (df_telemetry["equipment_id"] == wo["equipment_id"])
            & (df_telemetry["event_timestamp"] >= wo["wo_start_time"])
            & (df_telemetry["event_timestamp"] <= wo["wo_end_time"])
        )
        df_telemetry.loc[mask, "is_in_active_wo_window"] = True

    # Audit Flag: Telematics showing active fuel burn during a registered
    df_telemetry["source_discrepancy_flag"] = np.where(
        (df_telemetry["is_in_active_wo_window"])
        & (df_telemetry["fuel_rate_lph"] > 10.0),
        "DISCREPANCY: Active Burn During WO Downtime",
        "VERIFIED",
    )

    discrepancy_count = (
        df_telemetry["source_discrepancy_flag"] != "VERIFIED"
    ).sum()
    print(
        f"[✓] Source Reconciliation: Identified {discrepancy_count} audit
    )
```

```python
    # ---------------------------------------------------------------------
    # Export Clean Fact & Dimension Tables directly to Excel (.xlsx)
    # ---------------------------------------------------------------------
    # Fact 1: Clean Telematics
    df_telemetry[
        [
            "event_timestamp",
            "equipment_id",
            "site_location",
            "engine_temp_c",
            "hydraulic_pressure_bar",
            "fuel_rate_lph",
            "is_in_active_wo_window",
            "source_discrepancy_flag",
        ]
    ].to_excel("fact_fleet_telematics_clean.xlsx", index=False)

    # Fact 2: Clean SAP Work Orders
    df_wo[
        [
            "work_order_id",
            "equipment_id",
            "wo_start_time",
            "wo_end_time",
            "downtime_hours_logged",
            "maintenance_type",
            "parsed_component_category",
            "technician_notes",
        ]
    ].to_excel("fact_sap_work_orders_clean.xlsx", index=False)

    # Dimension: Equipment Master
    dim_equipment = (
        df_telemetry[["equipment_id", "model", "site_location"]]
        .drop_duplicates()
        .reset_index(drop=True)
    )
    dim_equipment.to_excel("dim_equipment.xlsx", index=False)

    print("\n=== Data Cleansing Complete! Excel Files Exported: ===")
    print("1. 'fact_fleet_telematics_clean.xlsx'")
    print("2. 'fact_sap_work_orders_clean.xlsx'")
    print("3. 'dim_equipment.xlsx'")

if __name__ == "__main__":
    clean_and_transform_fleet_data()
```

```
=== Starting Enterprise Fleet Data Cleansing Pipeline ===
[✓] Deduplication: Removed 45 network burst duplicate pings.
[✓] Unit Standardization: Converted all Imperial (Fahrenheit/PSI) metrics
to Metric (Celsius/Bar).
[✓] Anomaly Handling: Filtered out -9999 sensor glitches and interpolated
missing signal values.
[✓] NLP Text Parsing: Extracted structured component categories from fre
e-text technician logs.
[✓] Source Reconciliation: Identified 73 audit discrepancies between IoT
telematics and SAP logs.

=== Data Cleansing Complete! Excel Files Exported: ===
1. 'fact_fleet_telematics_clean.xlsx'
2. 'fact_sap_work_orders_clean.xlsx'
3. 'dim_equipment.xlsx'
```

In [ ]: